

13b_recheck_inverse_design_with_distance_blends

April 28, 2026

1 13b — Recheck Inverse Design with Distance Blend

This notebook verifies whether the final inverse-design scenarios remain valid if we choose a controlled forward blend for the distance targets.

Context:

- `prediction_submission.csv` is the conservative ExtraTrees final forward model.
- `prediction_submission_hybrid_mlp_distance_w_0.65.csv` is the aggressive hybrid MLP-distance candidate.
- `prediction_submission_controlled_blend_alpha_0.25.csv` and `alpha_0.40.csv` are safer test-set blends.

For inverse design, we need to make sure the 20 proposed scenarios still satisfy:

P80 ≤ 101

R95 ≤ 175

This notebook does **not** overwrite any official submission. It only creates diagnostics.

1.1 1. Imports and paths

```
[1]: from pathlib import Path
import json
import warnings
import joblib
import numpy as np
import pandas as pd

from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPRegressor

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 180)
```

```

PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.
    cwd()
FORWARD_DIR = PROJECT_ROOT / "data" / "raw" / "forward_prediction"
INVERSE_DIR = PROJECT_ROOT / "data" / "raw" / "inverse_design"
OUTPUTS_DIR = PROJECT_ROOT / "outputs"
SUBMISSIONS_DIR = OUTPUTS_DIR / "submissions"
MODELS_DIR = OUTPUTS_DIR / "models"
REPORTS_DIR = PROJECT_ROOT / "reports"

for p in [SUBMISSIONS_DIR, MODELS_DIR, REPORTS_DIR]:
    p.mkdir(parents=True, exist_ok=True)

print("Project root:", PROJECT_ROOT)
print("Forward dir:", FORWARD_DIR)
print("Inverse dir:", INVERSE_DIR)
print("Submissions dir:", SUBMISSIONS_DIR)
print("Models dir:", MODELS_DIR)
print("Reports dir:", REPORTS_DIR)

```

```

Project root: /home/alouiyaz/projects/boom-challenge-ejecta-prediction
Forward dir: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/data/raw/forward_prediction
Inverse dir: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/data/raw/inverse_design
Submissions dir: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/submissions
Models dir: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/outputs/models
Reports dir: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/reports

```

1.2 2. Load data, constraints, and current inverse design

```

[2]: input_cols = [
    "energy", "angle_rad", "coupling", "strength",
    "porosity", "gravity", "atmosphere", "shape_factor",
]

target_cols = [
    "P80", "fines_frac", "oversize_frac",
    "R95", "R50_fines", "R50_oversize",
]

fragmentation_targets = ["P80", "fines_frac", "oversize_frac"]
distance_targets = ["R95", "R50_fines", "R50_oversize"]

raw_train = pd.read_csv(FORWARD_DIR / "train.csv")[input_cols]
y = pd.read_csv(FORWARD_DIR / "train_labels.csv")[target_cols]

```

```

constraints_path = INVERSE_DIR / "constraints.json"
if not constraints_path.exists():
    constraints_path = PROJECT_ROOT / "constraints.json"

with open(constraints_path, "r") as f:
    constraints_data = json.load(f)

output_constraints = constraints_data["constraints"]
input_bounds = constraints_data["input_bounds"]

p80_min = output_constraints["p80_min"]
p80_max = output_constraints["p80_max"]
r95_max = output_constraints["r95_max"]

# Current final inverse-design file.
design_path = SUBMISSIONS_DIR / "design_submission.csv"
if not design_path.exists():
    raise FileNotFoundError(f"Missing inverse design file: {design_path}")

design_submission = pd.read_csv(design_path)

design_inputs = design_submission[input_cols].copy()

print("Training data:", raw_train.shape)
print("Targets:", y.shape)
print("Design submission:", design_submission.shape)
print("Constraints:")
display(pd.DataFrame([output_constraints]))
print("Input bounds:")
display(pd.DataFrame(input_bounds).T)
display(design_submission.head())

```

Training data: (2930, 8)

Targets: (2930, 6)

Design submission: (20, 9)

Constraints:

	p80_min	p80_max	r95_max
0	96.0	101.0	175.0

Input bounds:

	min	max
energy	0.500000	5.000000
angle_rad	0.261799	1.570796
coupling	0.200000	1.700000
strength	0.400000	4.200000
porosity	0.000000	0.330000

```
gravity      1.020000  10.470000
atmosphere   0.000000   1.000000
shape_factor 0.700000   1.500000
```

```
      submission_id  energy  angle_rad  coupling  strength  porosity  gravity
↪atmosphere shape_factor
0      0      3.598016   0.797647   1.291533   2.024534   0.304489   9.851536
↪ 0.422767      1.303159
1      1      3.219857   0.653336   0.937745   1.371766   0.268351  10.031512
↪ 0.819009      0.775107
2      2      3.883410   0.550208   0.754591   1.428060   0.294039   9.893283
↪ 0.800587      0.798041
3      3      2.769840   0.856038   0.933836   1.186456   0.266355   9.856101
↪ 0.429999      1.214199
4      4      3.491508   0.545414   1.620353   2.249113   0.264882   9.960773
↪ 0.715197      0.816463
```

1.3 3. Model class definitions

The saved joblib models need the same class definitions used in notebooks 09 and 12.

```
[3]: class PhysicsFeatureEngineer(BaseEstimator, TransformerMixin):
      def __init__(self, use_advanced: bool = False):
          self.use_advanced = use_advanced

      def fit(self, X, y=None):
          return self

      def transform(self, X):
          X = pd.DataFrame(X).copy()
          X = X[input_cols].copy()
          eps = 1e-9

          X["effective_energy"] = X["energy"] * X["coupling"]
          X["log_energy"] = np.log1p(X["energy"])
          X["log_effective_energy"] = np.log1p(X["effective_energy"])

          X["sin_angle"] = np.sin(X["angle_rad"])
          X["cos_angle"] = np.cos(X["angle_rad"])
          X["tan_angle"] = np.tan(X["angle_rad"])
          X["horizontal_energy"] = X["effective_energy"] * X["cos_angle"]
          X["vertical_energy"] = X["effective_energy"] * X["sin_angle"]
          X["vertical_horizontal_ratio"] = X["vertical_energy"] /
↪(X["horizontal_energy"] + eps)

          X["energy_per_strength"] = X["energy"] / (X["strength"] + eps)
          X["effective_energy_per_strength"] = X["effective_energy"] /
↪(X["strength"] + eps)
```

```

X["material_resistance_index"] = X["strength"] * (1 - X["porosity"])
X["fragmentation_index"] = X["effective_energy"] * X["porosity"] /  $\square$ 
 $\hookrightarrow$ (X["strength"] + eps)
X["coupling_porosity"] = X["coupling"] * X["porosity"]
X["coupling_atmosphere"] = X["coupling"] * X["atmosphere"]
X["porosity_strength"] = X["porosity"] * X["strength"]

X["energy_per_gravity"] = X["energy"] / (X["gravity"] + eps)
X["effective_energy_per_gravity"] = X["effective_energy"] /  $\square$ 
 $\hookrightarrow$ (X["gravity"] + eps)
X["horizontal_energy_per_gravity"] = X["horizontal_energy"] /  $\square$ 
 $\hookrightarrow$ (X["gravity"] + eps)
X["vertical_energy_per_gravity"] = X["vertical_energy"] / (X["gravity"]  $\square$ 
 $\hookrightarrow$ + eps)

X["drag_proxy"] = X["atmosphere"] * X["shape_factor"]
X["drag_per_gravity"] = X["drag_proxy"] / (X["gravity"] + eps)
X["atmosphere_shape_energy"] = X["atmosphere"] * X["shape_factor"] *  $\square$ 
 $\hookrightarrow$ X["effective_energy"]
X["atmosphere_per_gravity"] = X["atmosphere"] / (X["gravity"] + eps)

X["pi_gravity_proxy"] = (X["gravity"] * X["coupling"]) / (X["energy"] +  $\square$ 
 $\hookrightarrow$ eps)
X["pi_strength_proxy"] = X["strength"] / (X["gravity"] * X["coupling"]  $\square$ 
 $\hookrightarrow$ + eps)
X["pi_atmosphere_proxy"] = X["atmosphere"] / (X["gravity"] *  $\square$ 
 $\hookrightarrow$ X["coupling"] + eps)

X["porosity_regime"] = (X["porosity"] > 0.15).astype(int)
X["strength_regime"] = (X["strength"] > 2.6).astype(int)
X["angle_regime"] = (X["angle_rad"] > 0.95).astype(int)
X["atm_regime"] = (X["atmosphere"] > 0.30).astype(int)
X["regime_combo"] = (
    X["porosity_regime"] * 8
    + X["strength_regime"] * 4
    + X["angle_regime"] * 2
    + X["atm_regime"]
)

X["scaled_energy"] = X["effective_energy"] / (X["strength"] * np.
 $\hookrightarrow$ sqrt(X["gravity"])) + eps)
X["fragility"] = X["porosity"] / (X["strength"] + eps)
X["range_proxy"] = (X["effective_energy"] * (X["cos_angle"] ** 2)) / (
    X["gravity"] * X["strength"] + eps
)
X["energy_sin_angle"] = X["energy"] * X["sin_angle"]

```

```

        X["momentum_proxy"] = X["effective_energy"] * X["sin_angle"]
        X["coupling_per_atm_clipped"] = X["coupling"] / (X["atmosphere"] + 1e-3)
        X["log_coupling_per_atm"] = np.log1p(X["coupling_per_atm_clipped"])
        X["retention_factor"] = X["atmosphere"] * X["drag_proxy"] /
↪(X["energy"] + eps)

        if self.use_advanced:
            X["froude_proxy"] = np.sqrt(X["effective_energy"] / (X["gravity"] +
↪eps))
            X["stress_ratio_compact"] = X["effective_energy"] /
↪(X["material_resistance_index"] + eps)
            X["sqrt_effective_energy_per_strength"] = np.sqrt(
                X["effective_energy_per_strength"].clip(lower=0)
            )
            X["sqrt_effective_energy_per_gravity"] = np.sqrt(
                X["effective_energy_per_gravity"].clip(lower=0)
            )

        return X

class ColumnSelector(BaseEstimator, TransformerMixin):
    def __init__(self, columns):
        self.columns = list(columns)

    def fit(self, X, y=None):
        missing = [c for c in self.columns if c not in X.columns]
        if missing:
            raise ValueError(f"Missing columns: {missing}")
        return self

    def transform(self, X):
        return X[self.columns].copy()

def build_extratrees(random_state=42, n_estimators=800):
    return ExtraTreesRegressor(
        n_estimators=n_estimators,
        max_features="sqrt",
        min_samples_leaf=2,
        random_state=random_state,
        n_jobs=-1,
    )

def clip_predictions(preds, target):
    preds = np.asarray(preds).copy()

```

```

if target in ["fines_frac", "oversize_frac"]:
    return np.clip(preds, 0, 1)
return np.clip(preds, 0, None)

```

```

[4]: raw_features = input_cols.copy()

fragmentation_features_v1 = raw_features + [
    "effective_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "horizontal_energy", "vertical_energy",
    "energy_per_strength", "effective_energy_per_strength",
    "material_resistance_index", "fragmentation_index",
    "coupling_porosity", "coupling_atmosphere", "porosity_strength",
    "drag_proxy", "atmosphere_shape_energy",
    "pi_strength_proxy", "pi_atmosphere_proxy", "scaled_energy", "fragility",
    "energy_sin_angle", "momentum_proxy",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",
    ↪ "regime_combo",
]

fragmentation_features_v2 = fragmentation_features_v1 + [
    "stress_ratio_compact",
    "sqrt_effective_energy_per_strength",
]

distance_features_v1 = raw_features + [
    "effective_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "horizontal_energy", "vertical_energy",
    "energy_per_gravity", "effective_energy_per_gravity",
    "horizontal_energy_per_gravity", "vertical_energy_per_gravity",
    "drag_proxy", "drag_per_gravity", "atmosphere_shape_energy",
    ↪ "atmosphere_per_gravity",
    "pi_gravity_proxy", "pi_atmosphere_proxy", "scaled_energy", "range_proxy",
    "energy_sin_angle", "momentum_proxy",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",
    ↪ "regime_combo",
]

final_target_config = {
    "P80": {"feature_version": "v2", "features": fragmentation_features_v2},
    "fines_frac": {"feature_version": "v2", "features":
    ↪ fragmentation_features_v2},
    "oversize_frac": {"feature_version": "v2", "features":
    ↪ fragmentation_features_v2},
    "R95": {"feature_version": "v1", "features": distance_features_v1},
    "R50_fines": {"feature_version": "v1", "features": distance_features_v1},
    "R50_oversize": {"feature_version": "v1", "features": distance_features_v1},
}

```

```

def build_target_et_pipeline(target, random_state=42, n_estimators=800):
    cfg = final_target_config[target]
    use_advanced = cfg["feature_version"] == "v2"
    return Pipeline(steps=[
        ("features", PhysicsFeatureEngineer(use_advanced=use_advanced)),
        ("select", ColumnSelector(cfg["features"])),
        ("model", build_extratrees(random_state=random_state,
    ↪n_estimators=n_estimators)),
    ])

def build_mlp(random_state=42):
    return Pipeline(steps=[
        ("scaler", StandardScaler()),
        ("mlp", MLPRegressor(
            hidden_layer_sizes=(128, 64),
            activation="relu",
            solver="adam",
            alpha=1e-3,
            learning_rate_init=1e-3,
            max_iter=1200,
            early_stopping=True,
            validation_fraction=0.15,
            n_iter_no_change=40,
            random_state=random_state,
        )),
    ])

def build_feature_matrix_for_target(X_raw, target):
    cfg = final_target_config[target]
    X_fe = PhysicsFeatureEngineer(use_advanced=(cfg["feature_version"] ==
    ↪"v2")).fit_transform(X_raw)
    return X_fe[cfg["features"]].copy()

class FinalForwardPipeline:
    def __init__(self, target_config, random_state=42, n_estimators=800):
        self.target_config = target_config
        self.random_state = random_state
        self.n_estimators = n_estimators
        self.pipelines_ = {}

    def fit(self, X_raw, y_df):
        for i, target in enumerate(target_cols):

```



```

        pipe = build_target_et_pipeline(
            target,
            random_state=self.random_state + i,
            n_estimators=self.n_estimators,
        )
        pipe.fit(X_raw, y_df[target])
        self.pipelines_[target] = pipe
    return self

def predict(self, X_raw):
    preds = pd.DataFrame(index=X_raw.index)
    for target in target_cols:
        pred = self.pipelines_[target].predict(X_raw)
        preds[target] = clip_predictions(pred, target)
    return preds[target_cols]

class FinalHybridDistanceModel:
    def __init__(self, mlp_weight=0.0, random_state=42, n_estimators=800):
        self.mlp_weight = float(mlp_weight)
        self.random_state = random_state
        self.n_estimators = n_estimators
        self.et_pipelines_ = {}
        self.mlp_distance_models_ = {}

    def fit(self, X_raw, y_df):
        self.et_pipelines_ = {}
        self.mlp_distance_models_ = {}

        for i, target in enumerate(target_cols):
            pipe = build_target_et_pipeline(
                target,
                random_state=self.random_state + i,
                n_estimators=self.n_estimators,
            )
            pipe.fit(X_raw, y_df[target])
            self.et_pipelines_[target] = pipe

        if self.mlp_weight > 0:
            for i, target in enumerate(distance_targets):
                X_target = build_feature_matrix_for_target(X_raw, target)
                mlp = build_mlp(random_state=self.random_state + 100 + i)
                mlp.fit(X_target, y_df[target])
                self.mlp_distance_models_[target] = mlp

    return self

```

```

def predict(self, X_raw):
    preds = pd.DataFrame(index=X_raw.index)

    for target in target_cols:
        et_pred = self.et_pipelines_[target].predict(X_raw)
        preds[target] = clip_predictions(et_pred, target)

    if self.mlp_weight > 0:
        for target in distance_targets:
            X_target = build_feature_matrix_for_target(X_raw, target)
            mlp_pred = self.mlp_distance_models_[target].predict(X_target)
            blended = (1 - self.mlp_weight) * preds[target].values + self.
↪ mlp_weight * mlp_pred
            preds[target] = clip_predictions(blended, target)

    return preds[target_cols]

```

1.4 4. Load or rebuild base and hybrid forward models

The base model is the ExtraTrees final pipeline from notebook 09. The hybrid model is the MLP-distance model from notebook 12 with `mlp_weight = 0.65`.

```

[5]: base_model_path = MODELS_DIR / "final_forward_pipeline.joblib"
    hybrid_model_path = MODELS_DIR / "final_hybrid_mlp_distance_w_0.65.joblib"

    if base_model_path.exists():
        base_model = joblib.load(base_model_path)
        print("Loaded base model:", base_model_path)
    else:
        print("Base model not found. Rebuilding from training data.")
        base_model = FinalForwardPipeline(final_target_config, random_state=42,
↪ n_estimators=800)
        base_model.fit(raw_train, y)

    if hybrid_model_path.exists():
        hybrid_model = joblib.load(hybrid_model_path)
        print("Loaded hybrid model:", hybrid_model_path)
    else:
        print("Hybrid model not found. Rebuilding from training data.")
        hybrid_model = FinalHybridDistanceModel(mlp_weight=0.65, random_state=42,
↪ n_estimators=800)
        hybrid_model.fit(raw_train, y)

```

Loaded base model: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/models/final_forward_pipeline.joblib
 Loaded hybrid model: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/models/final_hybrid_mlp_distance_w_0.65.joblib

1.5 5. Predict inverse-design scenarios under each forward model

```
[6]: base_preds = base_model.predict(design_inputs).add_prefix("base_")
hybrid_preds = hybrid_model.predict(design_inputs).add_prefix("hybrid_")

recheck = pd.concat([design_submission, base_preds, hybrid_preds], axis=1)

# Controlled blends on predicted outputs for the 20 inverse-design scenarios.
for alpha in [0.25, 0.40]:
    for target in target_cols:
        if target in fragmentation_targets:
            recheck[f"alpha{int(alpha*100)}_{target}"] =
↪recheck[f"base_{target}"]
        else:
            recheck[f"alpha{int(alpha*100)}_{target}"] = (
                (1 - alpha) * recheck[f"base_{target}"]
                + alpha * recheck[f"hybrid_{target}"]
            )
            recheck[f"alpha{int(alpha*100)}_{target}"] =
↪recheck[f"alpha{int(alpha*100)}_{target}"].clip(lower=0)

print("Recheck table shape:", recheck.shape)
display(recheck.head())
```

Recheck table shape: (20, 33)

	submission_id	energy	angle_rad	coupling	strength	porosity	gravity
↪	atmosphere	shape_factor	base_P80	base_fines_frac	base_oversize_frac		
↪	base_R95	\					
0	0	3.598016	0.797647	1.291533	2.024534	0.304489	9.851536
↪	0.422767	1.303159	98.663679		0.093206		0.072531
↪	973302						88.
1	1	3.219857	0.653336	0.937745	1.371766	0.268351	10.031512
↪	0.819009	0.775107	98.648438		0.084552		0.072528
↪	952720						85.
2	2	3.883410	0.550208	0.754591	1.428060	0.294039	9.893283
↪	0.800587	0.798041	99.442042		0.077125		0.074867
↪	115577						77.
3	3	2.769840	0.856038	0.933836	1.186456	0.266355	9.856101
↪	0.429999	1.214199	97.907049		0.086700		0.062362
↪	647131						81.
4	4	3.491508	0.545414	1.620353	2.249113	0.264882	9.960773
↪	0.715197	0.816463	97.735768		0.099837		0.071675
↪	307290						83.
	base_R50_fines	base_R50_oversize	hybrid_P80	hybrid_fines_frac			
↪	hybrid_oversize_frac	hybrid_R95	hybrid_R50_fines	hybrid_R50_oversize			
↪	alpha25_P80	alpha25_fines_frac	\				

0	86.308420	38.441772	98.663679	0.093206	
	↪0.072531	90.200049	85.412697	37.607795	98.663679
	↪	0.093206			
1	83.312234	36.251774	98.648438	0.084552	
	↪0.072528	83.265891	80.145406	36.953548	98.648438
	↪	0.084552			
2	73.694526	32.081176	99.442042	0.077125	
	↪0.074867	82.473864	77.600209	36.597609	99.442042
	↪	0.077125			
3	81.902085	35.894790	97.907049	0.086700	
	↪0.062362	80.347309	83.964264	35.768614	97.907049
	↪	0.086700			
4	76.578171	32.180152	97.735768	0.099837	
	↪0.071675	87.631624	76.223228	36.265539	97.735768
	↪	0.099837			

	alpha25_oversize_frac	alpha25_R95	alpha25_R50_fines	alpha25_R50_oversize	
	↪alpha40_P80	alpha40_fines_frac	alpha40_oversize_frac	alpha40_R95	
	↪alpha40_R50_fines	\			
0	0.072531	89.279989	86.084489	38.233278	
	↪ 98.663679	0.093206	0.072531	89.464001	85.
	↪950131				
1	0.072528	85.281013	82.520527	36.427217	
	↪ 98.648438	0.084552	0.072528	84.877988	82.
	↪045503				
2	0.074867	78.455149	74.670947	33.210284	
	↪ 99.442042	0.077125	0.074867	79.258892	75.
	↪256799				
3	0.062362	81.322175	82.417630	35.863246	
	↪ 97.907049	0.086700	0.062362	81.127202	82.
	↪726956				
4	0.071675	84.388374	76.489435	33.201499	
	↪ 97.735768	0.099837	0.071675	85.037024	76.
	↪436193				

	alpha40_R50_oversize
0	38.108182
1	36.532483
2	33.887749
3	35.844320
4	33.814307

1.6 6. Feasibility check

A design is valid when:

96 <= P80 <= 101

R95 <= 175

```
[7]: def is_feasible_from_prefix(df, prefix):
    return df[f"{prefix}_P80"].between(p80_min, p80_max) & (df[f"{prefix}_R95"]
    ↪ <= r95_max)

for prefix in ["base", "hybrid", "alpha25", "alpha40"]:
    recheck[f"{prefix}_feasible"] = is_feasible_from_prefix(recheck, prefix)

feasibility_summary = pd.DataFrame([
    {
        "model": prefix,
        "n_feasible": int(recheck[f"{prefix}_feasible"].sum()),
        "n_total": len(recheck),
        "all_feasible": bool(recheck[f"{prefix}_feasible"].all()),
        "P80_min": recheck[f"{prefix}_P80"].min(),
        "P80_mean": recheck[f"{prefix}_P80"].mean(),
        "P80_max": recheck[f"{prefix}_P80"].max(),
        "R95_min": recheck[f"{prefix}_R95"].min(),
        "R95_mean": recheck[f"{prefix}_R95"].mean(),
        "R95_max": recheck[f"{prefix}_R95"].max(),
        "R95_margin_min": r95_max - recheck[f"{prefix}_R95"].max(),
    }
    for prefix in ["base", "hybrid", "alpha25", "alpha40"]
])

display(feasibility_summary)

# Rows that fail in any scenario.
failed_any = recheck[
    ~(recheck["base_feasible"] & recheck["hybrid_feasible"] &
    ↪ recheck["alpha25_feasible"] & recheck["alpha40_feasible"])
].copy()

print("Rows failing under at least one forward variant:", len(failed_any))
if len(failed_any):
    display(failed_any[[
        "submission_id", *input_cols,
        "base_P80", "base_R95", "base_feasible",
        "hybrid_P80", "hybrid_R95", "hybrid_feasible",
        "alpha25_P80", "alpha25_R95", "alpha25_feasible",
        "alpha40_P80", "alpha40_R95", "alpha40_feasible",
    ]])
```

	model	n_feasible	n_total	all_feasible	P80_min	P80_mean	P80_max
↪	R95_min	R95_mean	R95_max	R95_margin_min			
0	base	20	20	True	97.362165	98.520519	99.709877
↪	74.922776	85.931067	101.316388	73.683612			

1	hybrid	20	20	True	97.362165	98.520519	99.709877	↳ 78.624967	84.734990	97.115946	77.884054
2	alpha25	20	20	True	97.362165	98.520519	99.709877	↳ 76.039612	85.632048	100.266277	74.733723
3	alpha40	20	20	True	97.362165	98.520519	99.709877	↳ 76.709713	85.452636	99.636211	75.363789

Rows failing under at least one forward variant: 0

1.7 7. Detailed diagnostics for selected designs

```
[8]: cols_to_show = [
    "submission_id", *input_cols,
    "base_P80", "base_R95", "base_R50_fines", "base_R50_oversize",
    "hybrid_P80", "hybrid_R95", "hybrid_R50_fines", "hybrid_R50_oversize",
    "alpha25_P80", "alpha25_R95", "alpha25_R50_fines", "alpha25_R50_oversize",
    "alpha40_P80", "alpha40_R95", "alpha40_R50_fines", "alpha40_R50_oversize",
    "base_feasible", "hybrid_feasible", "alpha25_feasible", "alpha40_feasible",
]
```

```
display(recheck[cols_to_show])
```

```
summary_rows = []
for prefix in ["base", "hybrid", "alpha25", "alpha40"]:
    for target in target_cols:
        summary_rows.append({
            "model": prefix,
            "target": target,
            "mean": recheck[f"{prefix}_{target}"].mean(),
            "std": recheck[f"{prefix}_{target}"].std(),
            "min": recheck[f"{prefix}_{target}"].min(),
            "median": recheck[f"{prefix}_{target}"].median(),
            "max": recheck[f"{prefix}_{target}"].max(),
        })
```

```
selected_design_prediction_summary = pd.DataFrame(summary_rows)
display(selected_design_prediction_summary)
```

	submission_id	energy	angle_rad	coupling	strength	porosity	gravity
↳	atmosphere	shape_factor	base_P80	base_R95	base_R50_fines		
↳	base_R50_oversize	hybrid_P80	\				
0	0	3.598016	0.797647	1.291533	2.024534	0.304489	9.851536
↳	0.422767	1.303159	98.663679	88.973302	86.308420		38.
↳	441772	98.663679					
1	1	3.219857	0.653336	0.937745	1.371766	0.268351	10.031512
↳	0.819009	0.775107	98.648438	85.952720	83.312234		36.
↳	251774	98.648438					

2	2	3.883410	0.550208	0.754591	1.428060	0.294039	9.893283	┐
↪	0.800587	0.798041	99.442042	77.115577	73.694526		32.	
↪	081176	99.442042						
3	3	2.769840	0.856038	0.933836	1.186456	0.266355	9.856101	┐
↪	0.429999	1.214199	97.907049	81.647131	81.902085		35.	
↪	894790	97.907049						
4	4	3.491508	0.545414	1.620353	2.249113	0.264882	9.960773	┐
↪	0.715197	0.816463	97.735768	83.307290	76.578171		32.	
↪	180152	97.735768						
5	5	2.818390	0.657906	0.841158	1.143469	0.291200	9.963445	┐
↪	0.841676	0.994703	97.931658	80.121540	78.788268		38.	
↪	135962	97.931658						
6	6	3.944120	0.566188	0.829191	1.568268	0.293000	9.997341	┐
↪	0.808564	1.187302	98.327340	74.922776	72.164253		34.	
↪	042642	98.327340						
7	7	2.972409	0.772728	1.329016	1.663816	0.306724	9.953774	┐
↪	0.682885	0.850362	98.666910	84.670167	80.211657		36.	
↪	197564	98.666910						
8	8	3.639483	0.789794	0.795982	1.343432	0.251777	10.083053	┐
↪	0.864865	1.046821	98.185973	79.786165	77.261703		35.	
↪	388345	98.185973						
9	9	3.787539	0.728334	1.498127	2.416876	0.292087	9.921431	┐
↪	0.611923	0.918703	98.665610	91.801889	85.705823		37.	
↪	826083	98.665610						
10	10	3.166497	0.693846	1.314625	1.731167	0.271333	9.309159	┐
↪	0.747243	1.190337	98.831208	98.343473	91.895308		42.	
↪	550995	98.831208						
11	11	2.825667	0.583369	0.904537	1.189014	0.262987	9.981676	┐
↪	0.681155	1.225510	98.784140	81.271593	79.849082		38.	
↪	290429	98.784140						
12	12	2.809180	0.530423	1.155803	1.432078	0.286258	9.875173	┐
↪	0.671053	1.142482	97.475070	86.905404	83.232849		37.	
↪	148297	97.475070						
13	13	3.948492	0.670550	0.752039	1.465636	0.298577	9.923262	┐
↪	0.650624	1.035515	99.709877	86.863968	84.678929		39.	
↪	238340	99.709877						
14	14	3.166497	0.501950	1.314625	1.731167	0.271333	10.083053	┐
↪	0.864865	1.333853	98.472597	87.101933	80.411445		36.	
↪	203124	98.472597						
15	15	3.639483	0.693846	0.795982	1.343432	0.251777	10.083053	┐
↪	0.512000	1.333853	98.936222	87.735506	86.406049		38.	
↪	749995	98.936222						
16	16	3.166497	0.789794	1.314625	1.731167	0.271333	10.083053	┐
↪	0.512000	1.046821	98.459342	86.791318	86.569815		38.	
↪	192267	98.459342						

17	17	3.639483	0.693846	0.795982	1.343432	0.251777	10.083053	▢
↪	0.629622	1.190337	98.863927	84.276903		82.559993	37.	
↪	963994	98.863927						
18	18	3.166497	0.693846	1.314625	1.731167	0.251777	10.083053	▢
↪	0.512000	0.903305	99.341362	89.716301		87.197422	39.	
↪	306345	99.341362						
19	19	4.111200	0.732915	1.266471	2.273687	0.304414	9.929749	▢
↪	0.433913	0.845490	97.362165	101.316388		96.759174	42.	
↪	860981	97.362165						
	hybrid_R95	hybrid_R50_fines	hybrid_R50_oversize	alpha25_P80	alpha25_R95			▢
↪	alpha25_R50_fines	alpha25_R50_oversize	alpha40_P80	alpha40_R95				▢
↪	alpha40_R50_fines	\						
0	90.200049	85.412697		37.607795	98.663679	89.279989		▢
↪	86.084489		38.233278	98.663679	89.464001	85.		
↪	950131							
1	83.265891	80.145406		36.953548	98.648438	85.281013		▢
↪	82.520527		36.427217	98.648438	84.877988	82.		
↪	045503							
2	82.473864	77.600209		36.597609	99.442042	78.455149		▢
↪	74.670947		33.210284	99.442042	79.258892	75.		
↪	256799							
3	80.347309	83.964264		35.768614	97.907049	81.322175		▢
↪	82.417630		35.863246	97.907049	81.127202	82.		
↪	726956							
4	87.631624	76.223228		36.265539	97.735768	84.388374		▢
↪	76.489435		33.201499	97.735768	85.037024	76.		
↪	436193							
5	79.075659	75.392887		37.884757	97.931658	79.860070		▢
↪	77.939422		38.073161	97.931658	79.703187	77.		
↪	430115							
6	79.390119	81.057239		40.136199	98.327340	76.039612		▢
↪	74.387499		35.566031	98.327340	76.709713	75.		
↪	721447							
7	85.815183	80.059302		35.866655	98.666910	84.956421		▢
↪	80.173568		36.114837	98.666910	85.128173	80.		
↪	150715							
8	78.624967	80.002832		37.423306	98.185973	79.495866		▢
↪	77.946985		35.897085	98.185973	79.321686	78.		
↪	358154							
9	90.862435	81.202120		37.900956	98.665610	91.567025		▢
↪	84.579897		37.844801	98.665610	91.426107	83.		
↪	904342							
10	92.022746	86.261532		42.507638	98.831208	96.763291		▢
↪	90.486864		42.540156	98.831208	95.815182	89.		
↪	641798							

11	79.364031	77.217854	38.846267	98.784140	80.794702	↵
	↵ 79.191275	38.429389	98.784140	80.508568	78.	
	↵ 796591					
12	84.304302	84.085472	38.826564	97.475070	86.255128	↵
	↵ 83.446005	37.567864	97.475070	85.864963	83.	
	↵ 573898					
13	84.697418	86.743192	40.305307	99.709877	86.322331	↵
	↵ 85.194995	39.505082	99.709877	85.997348	85.	
	↵ 504635					
14	82.605640	77.736812	39.655227	98.472597	85.977860	↵
	↵ 79.742787	37.066150	98.472597	85.303416	79.	
	↵ 341592					
15	83.848057	87.285856	38.034922	98.936222	86.763643	↵
	↵ 86.626001	38.571227	98.936222	86.180526	86.	
	↵ 757972					
16	85.500475	81.170034	35.309183	98.459342	86.468608	↵
	↵ 85.219870	37.471496	98.459342	86.274981	84.	
	↵ 409903					
17	81.207024	82.943368	37.864246	98.863927	83.509433	↵
	↵ 82.655836	37.939057	98.863927	83.048952	82.	
	↵ 713343					
18	86.347061	81.080590	35.379975	99.341362	88.873991	↵
	↵ 85.668214	38.324752	99.341362	88.368605	84.	
	↵ 750689					
19	97.115946	90.749195	38.305967	97.362165	100.266277	↵
	↵ 95.256679	41.722227	97.362165	99.636211	94.	
	↵ 355182					

	alpha40_R50_oversize	base_feasible	hybrid_feasible	alpha25_feasible	↵
	↵ alpha40_feasible				
0	38.108182	True	True	True	↵
	↵ True				
1	36.532483	True	True	True	↵
	↵ True				
2	33.887749	True	True	True	↵
	↵ True				
3	35.844320	True	True	True	↵
	↵ True				
4	33.814307	True	True	True	↵
	↵ True				
5	38.035480	True	True	True	↵
	↵ True				
6	36.480065	True	True	True	↵
	↵ True				
7	36.065201	True	True	True	↵
	↵ True				

8		36.202330	True	True	True	␣
↪	True					
9		37.856032	True	True	True	␣
↪	True					
10		42.533652	True	True	True	␣
↪	True					
11		38.512764	True	True	True	␣
↪	True					
12		37.819604	True	True	True	␣
↪	True					
13		39.665127	True	True	True	␣
↪	True					
14		37.583965	True	True	True	␣
↪	True					
15		38.463966	True	True	True	␣
↪	True					
16		37.039033	True	True	True	␣
↪	True					
17		37.924095	True	True	True	␣
↪	True					
18		37.735797	True	True	True	␣
↪	True					
19		41.038975	True	True	True	␣
↪	True					

	model	target	mean	std	min	median	max
0	base	P80	98.520519	0.624189	97.362165	98.656059	99.709877
1	base	finest_frac	0.088494	0.007092	0.077125	0.089029	0.099837
2	base	oversize_frac	0.071152	0.003953	0.062362	0.071798	0.078685
3	base	R95	85.931067	6.397745	74.922776	86.372019	101.316388
4	base	R50_fines	82.774360	5.859489	72.164253	82.896421	96.759174
5	base	R50_oversize	37.347251	2.772646	32.081176	37.895038	42.860981
6	hybrid	P80	98.520519	0.624189	97.362165	98.656059	99.709877
7	hybrid	finest_frac	0.088494	0.007092	0.077125	0.089029	0.099837
8	hybrid	oversize_frac	0.071152	0.003953	0.062362	0.071798	0.078685
9	hybrid	R95	84.734990	4.908672	78.624967	84.076180	97.115946
10	hybrid	R50_fines	81.816704	4.083988	75.392887	81.125312	90.749195
11	hybrid	R50_oversize	37.872014	1.840141	35.309183	37.874501	42.507638
12	alpha25	P80	98.520519	0.624189	97.362165	98.656059	99.709877
13	alpha25	finest_frac	0.088494	0.007092	0.077125	0.089029	0.099837
14	alpha25	oversize_frac	0.071152	0.003953	0.062362	0.071798	0.078685
15	alpha25	R95	85.632048	5.907932	76.039612	85.629436	100.266277
16	alpha25	R50_fines	82.534946	5.206505	74.387499	82.588182	95.256679
17	alpha25	R50_oversize	37.478442	2.296924	33.201499	37.706333	42.540156
18	alpha40	P80	98.520519	0.624189	97.362165	98.656059	99.709877
19	alpha40	finest_frac	0.088494	0.007092	0.077125	0.089029	0.099837
20	alpha40	oversize_frac	0.071152	0.003953	0.062362	0.071798	0.078685

21	alpha40	R95	85.452636	5.645270	76.709713	85.215795	99.636211
22	alpha40	R50_fines	82.391298	4.865395	75.256799	82.720150	94.355182
23	alpha40	R50_oversize	37.557156	2.063120	33.814307	37.777700	42.533652

1.8 8. Save diagnostics

```
[9]: recheck_path = REPORTS_DIR / "inverse_design_recheck_with_distance_blends.csv"
summary_path = REPORTS_DIR / "inverse_design_recheck_summary.csv"

recheck.to_csv(recheck_path, index=False)
feasibility_summary.to_csv(summary_path, index=False)

print("Saved detailed recheck to:", recheck_path)
print("Saved summary to:", summary_path)
```

Saved detailed recheck to: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/reports/inverse_design_recheck_with_distance_blends.csv
 Saved summary to: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/reports/inverse_design_recheck_summary.csv

1.9 9. Decision notes

Interpretation guide:

- If all 20 scenarios remain feasible under **base**, **alpha25**, and **alpha40**, the current inverse design is robust.
- If **hybrid** fails but **alpha25** passes, this supports using the controlled **alpha25** forward submission instead of the aggressive hybrid.
- If **alpha40** passes and keeps enough R95 margin, it is a valid higher-risk candidate.
- If a design fails under **alpha25**, regenerate inverse design with the chosen forward blend before final submission.

Recommended final decision rule:

Forward: use **alpha25** if we want a prudent correction of ExtraTrees under OOD.

Inverse: keep **design_submission.csv** only if **alpha25** recheck confirms all 20 designs remain feasible.